Employee Management System

Submitted By

Student Name	Student ID
Md parvej	251-15-901
Md Abu Salih	251-15-100
Anil Shil Antu	251-15-667
Yeasin Arafat	251-15-914
Imranul Karim	251-15-575

MINI LAB PROJECT REPORT

This Report Presented in Partial Fulfillment of the course **CSE-222: Object Oriented Programming in the Computer Science and Engineering Department**

DAFFODIL INTERNATIONAL UNIVERSITY

Dhaka, Bangladesh

17/08/2026

DECLARATION

We hereby declare that this lab project has been done by us under the supervision of **Md. Taufik Hasan**, **Lecturer**, Department of Computer Science and Engineering, Daffodil International University. We also declare that neither this project nor any part of this project has been submitted elsewhere as a lab project.

Submitted To:

Md. Taufik Hasan
Lecturer
Department of Computer Science and Engineering,
Daffodil International University

COURSE & PROGRAM OUTCOME

The following course have course outcomes as following:.

Table 1: Course Outcome Statements

CO's	Statements
CO1	Understand and classify key **OOP concepts**, including **interfaces** and **abstract** classes.
CO2	Apply OOP principles to design and implement Java-based solutions for **real-world problems**.
CO3	Use **UML diagrams** to design and communicate software solutions.
CO4	Develop advanced Java applications using **exception handling, multi-threading, and networking.**
CO5	Solve a Complex Engineering Problem (CEP) through a comprehensive course project integrating **OOP concepts, UML**, and modern tools.

Table 2: Mapping of CO, PO, Blooms, KP and CEP

CO	PO	Blooms	KP	CEP / EP
CO1	PO1	C1, C2	KP3	EP1, EP3
CO2	PO2	C2, C4	KP3	EP1, EP3
CO3	PO3	C3, C6, P3	KP4	EP1, EP6
CO4	PO3	C4, C6, A3	KP4	EP1, EP3

The mapping justification of this table is provided in section **4.3.1**, **4.3.2** and **4.3.**

Table of Contents

Chapter 1
Introduction

This chapter introduces the Employee Management System (EMS) project, presenting the background, problem statement, motivation, objectives, feasibility analysis, gaps in existing solutions, and expected outcomes. The chapter establishes the foundation and context for the entire project.

1.1 Introduction

Managing human resources is a critical function for any organization, regardless of its size. Traditional or manual employee record-keeping systems often suffer from challenges such as data redundancy, slow retrieval times, and human error during data entry. For small to medium-scale organizations, deploying heavy, database-driven enterprise systems is often overkill, cost-prohibitive, and complex to maintain. This project develops a lightweight, comprehensive Employee Management System (EMS) that addresses these challenges by incorporating fundamental Object-Oriented Programming (OOP) principles and dynamic data structures. Built using Core Java and Java Swing, the system utilizes ArrayList for efficient in-memory data management (O(1) append, O(n) search and update), leverages polymorphism to handle different employee types (Permanent and Part-Time), and employs dynamic GUI layouts (CardLayout, GridBagLayout) for a seamless user experience.

1.2 Motivation

The primary motivation for this project arises from the desire to apply theoretical Object-Oriented Programming (OOP) concepts to a practical, real-world scenario. While modern systems heavily rely on complex relational databases, building an in-memory CRUD (Create, Read, Update, Delete) system using collections like ArrayList provides a deeper understanding of dynamic memory allocation and state management. The project bridges the gap between backend logic and frontend design, demonstrating how polymorphic behavior (handling Permanent vs. Part-Time employees) seamlessly integrates with dynamic graphical interfaces. Furthermore, the educational motivation includes understanding how strict input validation (using Regular Expressions and mathematical constraints) directly impacts data integrity, ensuring that a system remains robust without relying on external database constraints.

1.3 Objectives

- Implement a robust desktop application for managing workforce records using a modern Java Swing GUI.
- Develop core CRUD operations: register new employees, search by ID (O(n)), update records, and delete employees.
- Implement polymorphic data handling to accurately manage different employee types (Permanent employees with Base Salary/Bonus vs. Part-Time employees with Hours/Rate).
- Enable dynamic Type Conversion during updates (e.g., converting a Part-Time employee to Permanent) with automated backend object replacement.

- Build strict, real-time input validation systems (Regex for names, non-negative checks for numerical values) with inline visual error feedback.
- Design a responsive Graphical User Interface (GUI) utilizing CardLayout for dynamic form switching and GridBagLayout for precise component alignment.

1.4 Feasibility Study

Existing HR software like Workday, BambooHR, and various localized ERPs have demonstrated the critical need for digital workforce management. However, these are massive, cloud-based architectures. This project's feasibility as a lightweight, standalone alternative is exceptionally high because: (1) All required technologies (Core Java, Swing GUI, Collections framework) are built into the standard Java Development Kit (JDK) without requiring third-party libraries, (2) The scope is perfectly tailored for a desktop application with clear operational boundaries, (3) It eliminates the need for external database servers by utilizing an efficient in-memory ArrayList storage engine, making it highly portable, and (4) The architecture is cleanly modular, carefully separating the backend business logic (EmployeeManager) from the frontend UI frames.

1.5 Gap Analysis

Gap 1 - Practical OOP Implementation: Many academic projects teach Inheritance and Polymorphism merely as theoretical concepts. This project bridges that gap by practically applying these concepts to manage distinct PermanentEmployee and PartTimeEmployee objects under a unified Employee parent class.

Gap 2 - Integration of Logic and GUI: Educational projects often focus solely on console-based outputs. This project demonstrates how backend data structures actively synchronize with a graphical interface (e.g., JTable, dynamic form fields) in real-time.

Gap 3 - Data Integrity without External Databases: Modern applications rely heavily on SQL constraints for data integrity. This project demonstrates how to achieve strict data validation (preventing duplicate IDs, invalid characters, and negative salaries) purely through application-level logic and exception handling.

1.6 Project Outcome

- A fully functional, bug-free desktop Employee Management System with all core CRUD operations working accurately.
- Practical demonstration of OOP principles (Inheritance, Polymorphism, Abstraction) in a production-like environment.
- A polished, modern Java Swing GUI featuring dynamic layouts, placeholders, and intelligent inline error handling.
- A robust in-memory data management engine capable of dynamic type switching and real-time UI state synchronization.
- A comprehensive, well-structured codebase reflecting standard software engineering practices and clean architecture.

.

Chapter 2

Proposed Methodology/Architecture

This chapter details the system design, architecture, and methodology used to implement the Employee Management System (EMS). It covers requirement analysis, the proposed system architecture, design specifications, and the overall structural planning of the application.

2.1 Requirement Analysis & Design Specification

2.1.1 Overview

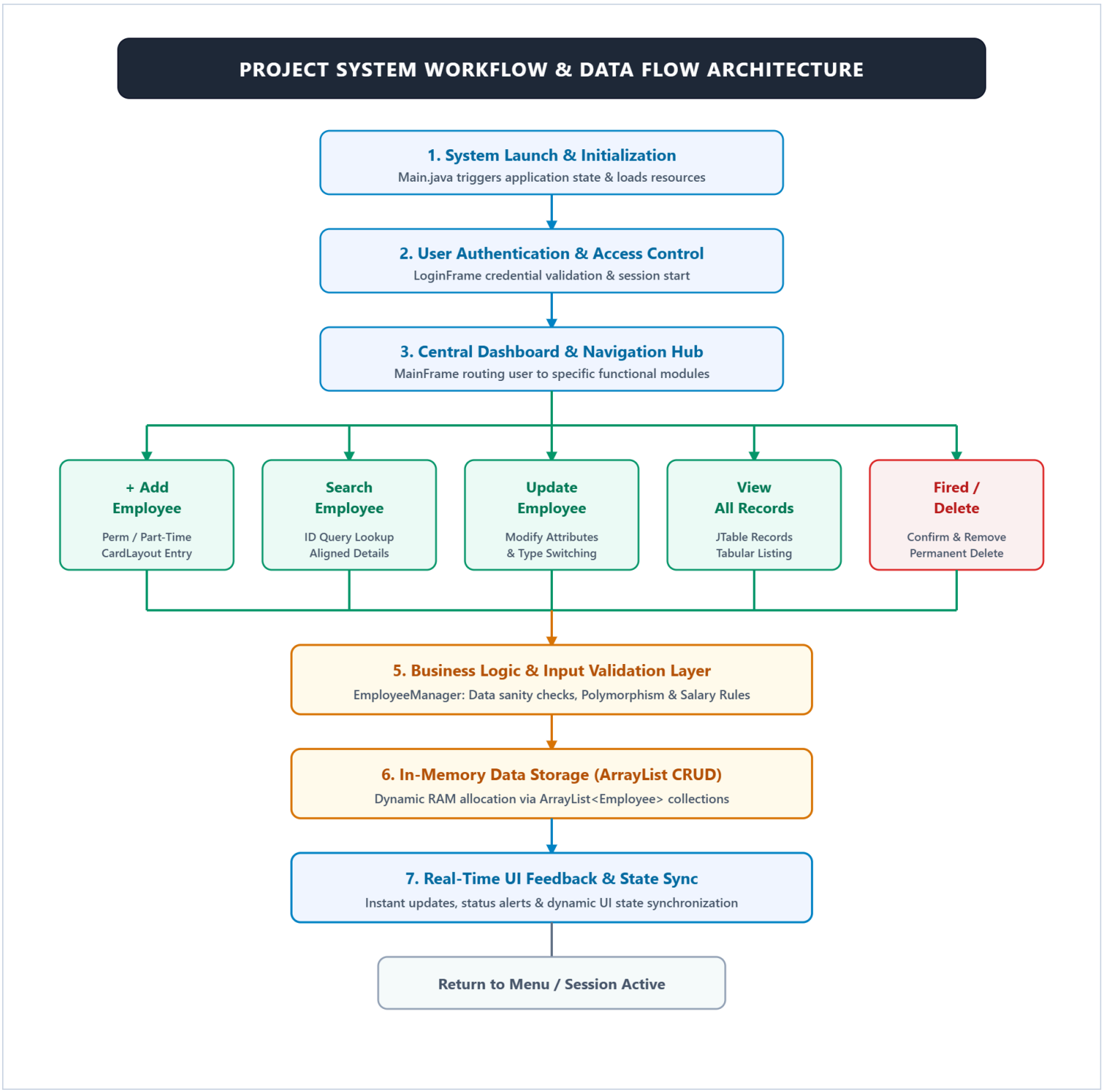
The proposed Employee Management System is designed to simulate modern human resource operations with a strict focus on data integrity, user-friendly interaction, and structural reliability. The system is primarily tailored for an administrative user (such as an HR Manager or System Admin), ensuring centralized control and streamlined management of workforce records without the overhead of a complex external database.

The system is capable of handling core administrative functionalities such as employee registration, dynamic data retrieval (search), seamless record modification, and secure deletion. In addition, it incorporates advanced functional features like polymorphic type conversion (allowing an administrator to dynamically switch an employee's role between Permanent and Part-Time), automated financial computations based on specific employee types, and real-time inline input validation to actively prevent erroneous data entry.

To ensure a robust system design, the requirements are divided into two distinct categories. Functional requirements define what the system fundamentally does, including the complete execution of CRUD (Create, Read, Update, Delete) operations and role-based salary calculations. Non-functional requirements dictate how the system performs, focusing heavily on graphical user interface (GUI) responsiveness, memory efficiency (utilizing Java ArrayList for O(1) append and O(n) traversal operations), strict data validation accuracy, and overall ease of use through modern structural layouts like CardLayout and GridBagLayout.

Overall, the system is expertly designed to balance frontend UI/UX usability, backend data integrity, and structural simplicity. It stands as a practical demonstration of how fundamental Object-Oriented Programming (OOP) principles and dynamic in-memory data structures can be seamlessly integrated into a functional, real-world desktop application.

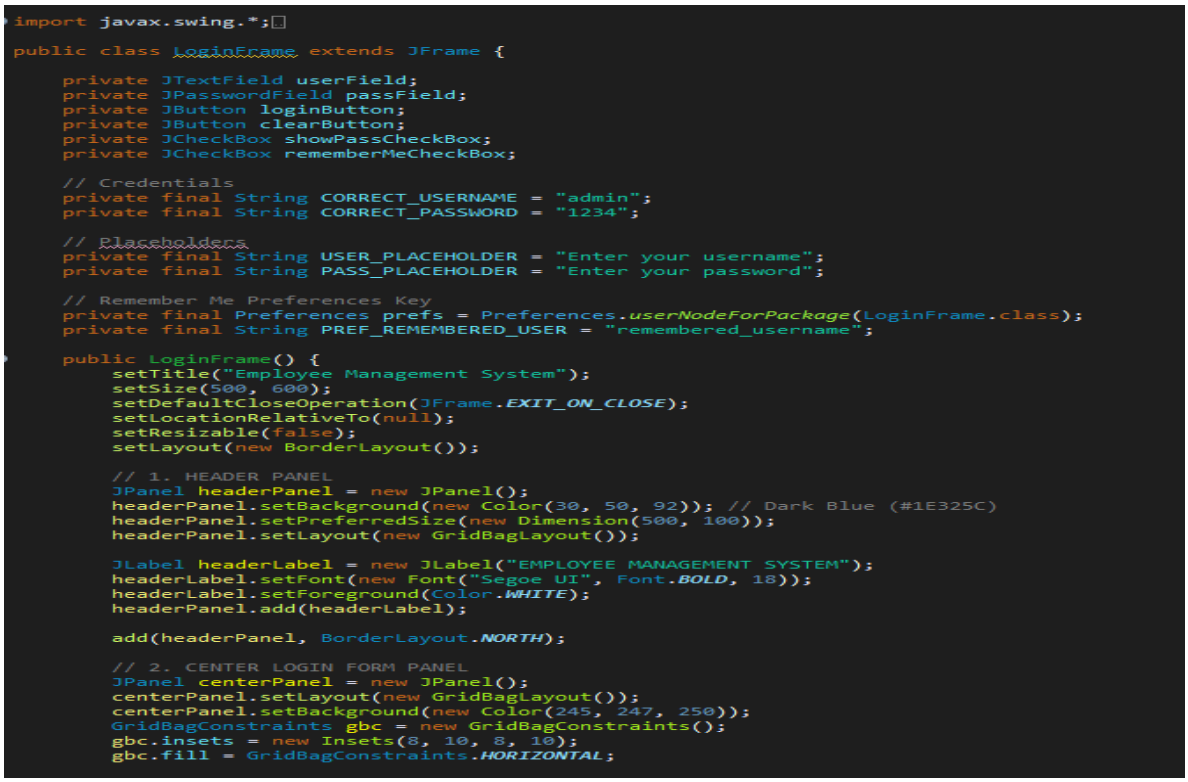
2.1.2 Proposed Methodology/ System Design



2.1.3 UI Design

System Login Menu

When the system starts, users see the main login menu:



```
import javax.swing.*;

public class LoginFrame extends JFrame {

    private JTextField userField;
    private JPasswordField passField;
    private JButton loginButton;
    private JButton clearButton;
    private JCheckBox showPassCheckBox;
    private JCheckBox rememberMeCheckBox;

    // Credentials
    private final String CORRECT_USERNAME = "admin";
    private final String CORRECT_PASSWORD = "1234";

    // Placeholders
    private final String USER_PLACEHOLDER = "Enter your username";
    private final String PASS_PLACEHOLDER = "Enter your password";

    // Remember Me Preferences Key
    private final Preferences prefs = Preferences.userNodeForPackage(LoginFrame.class);
    private final String PREF_REMEMBERED_USER = "remembered_username";

    public LoginFrame() {
        setTitle("Employee Management System");
        setSize(500, 600);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setResizable(false);
        setLayout(new BorderLayout());

        // 1. HEADER PANEL
        JPanel headerPanel = new JPanel();
        headerPanel.setBackground(new Color(30, 50, 92)); // Dark Blue (#1E325C)
        headerPanel.setPreferredSize(new Dimension(500, 100));
        headerPanel.setLayout(new GridBagLayout());

        JLabel headerLabel = new JLabel("EMPLOYEE MANAGEMENT SYSTEM");
        headerLabel.setFont(new Font("Segoe UI", Font.BOLD, 18));
        headerLabel.setForeground(Color.WHITE);
        headerPanel.add(headerLabel);

        add(headerPanel, BorderLayout.NORTH);

        // 2. CENTER LOGIN FORM PANEL
        JPanel centerPanel = new JPanel();
        centerPanel.setLayout(new GridBagLayout());
        centerPanel.setBackground(new Color(245, 247, 250));
        GridBagConstraints gbc = new GridBagConstraints();
        gbc.insets = new Insets(8, 10, 8, 10);
        gbc.fill = GridBagConstraints.HORIZONTAL;
```

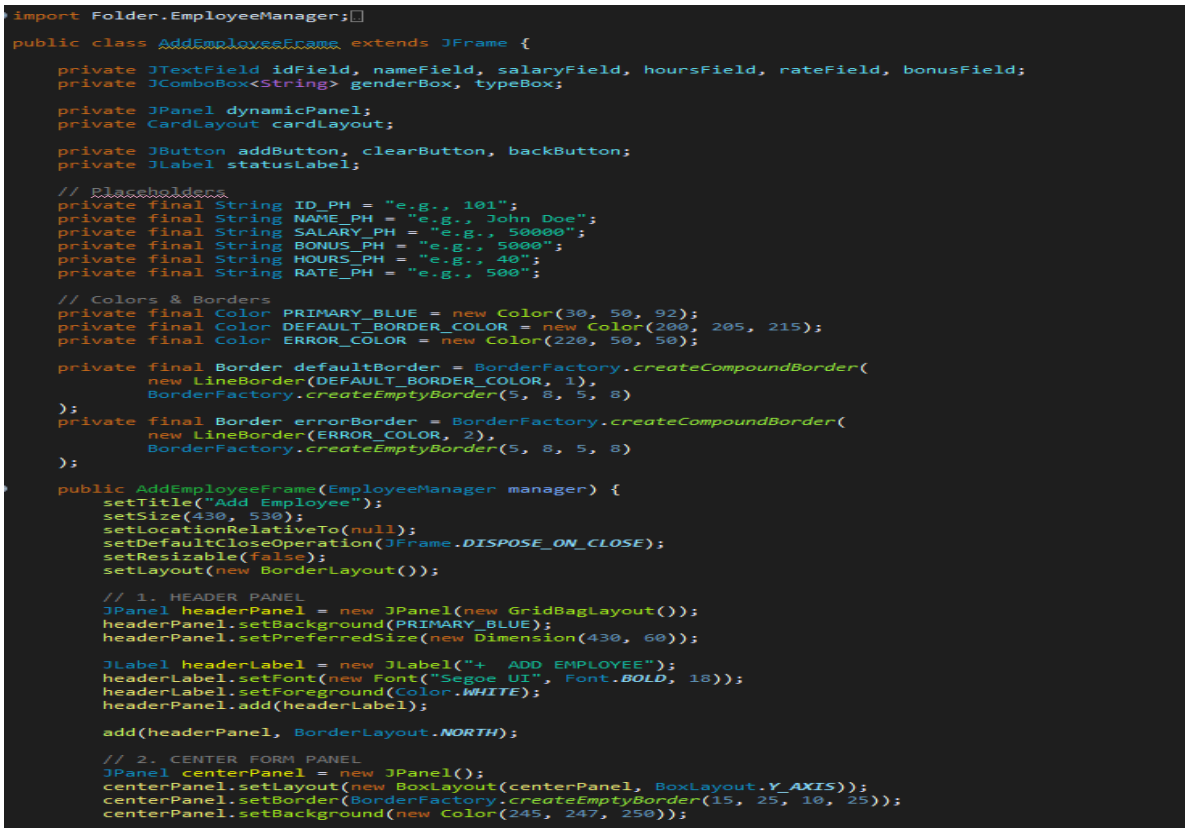
System Menu

After authentication, users can access comprehensive management features:

```
    public MainFrame() {
        this(new EmployeeManager()); // Default constructor creates new EmployeeManager
    }

    public MainFrame(EmployeeManager manager) {
        this.manager = manager;

        //============================
        // Window settings
        //============================
        setTitle("Employee Management System");
        setSize(460, 540);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);    // open in the center of the screen
        setLayout(new BorderLayout());

        //============================
        // Build the parts
        //============================
        add(buildHeader(), BorderLayout.NORTH);
        add(buildMenu(),   BorderLayout.CENTER);

        //============================
        // Show the window
        //============================
        setVisible(true);
    }

    //============================================================
    // HEADER PANEL
    //============================================================
    private JPanel buildHeader() {
        JPanel header = new JPanel();
        header.setBackground(DARK);
        header.setPreferredSize(new Dimension(0, 80));
        header.setLayout(new FlowLayout(FlowLayout.CENTER, 10, 22));

        JLabel title = new JLabel("▣  EMPLOYEE MANAGEMENT SYSTEM");
        title.setFont(new Font("Segoe UI Emoji", Font.BOLD, 20));
        title.setForeground(WHITE);

        header.add(title);
        return header;
    }

    //============================================================
    // MENU PANEL (the 6 buttons)
    //============================================================
    private JPanel buildMenu() {
        JPanel menu = new JPanel(new GridBagLayout());
        menu.setBackground(new Color(245, 247, 250));
        menu.setBorder(BorderFactory.createEmptyBorder(25, 40, 25, 40));

        GridBagConstraints gc = new GridBagConstraints();
        gc.gridx = 0;
        gc.fill = GridBagConstraints.HORIZONTAL;
```

User Employee Interface

User add Employee interface



```
import Folder.EmployeeManager;

public class AddEmployeeFrame extends JFrame {

    private JTextField idField, nameField, salaryField, hoursField, rateField, bonusField;
    private JComboBox<String> genderBox, typeBox;

    private JPanel dynamicPanel;
    private CardLayout cardLayout;

    private JButton addButton, clearButton, backButton;
    private JLabel statusLabel;

    // Placeholders
    private final String ID_PH = "e.g., 101";
    private final String NAME_PH = "e.g., John Doe";
    private final String SALARY_PH = "e.g., 50000";
    private final String BONUS_PH = "e.g., 5000";
    private final String HOURS_PH = "e.g., 40";
    private final String RATE_PH = "e.g., 500";

    // Colors & Borders
    private final Color PRIMARY_BLUE = new Color(30, 50, 92);
    private final Color DEFAULT_BORDER_COLOR = new Color(200, 205, 215);
    private final Color ERROR_COLOR = new Color(220, 50, 50);

    private final Border defaultBorder = BorderFactory.createCompoundBorder(
            new LineBorder(DEFAULT_BORDER_COLOR, 1),
            BorderFactory.createEmptyBorder(5, 8, 5, 8)
    );
    private final Border errorBorder = BorderFactory.createCompoundBorder(
            new LineBorder(ERROR_COLOR, 2),
            BorderFactory.createEmptyBorder(5, 8, 5, 8)
    );

    public AddEmployeeFrame(EmployeeManager manager) {
        setTitle("Add Employee");
        setSize(430, 530);
        setLocationRelativeTo(null);
        setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
        setResizable(false);
        setLayout(new BorderLayout());

        // 1. HEADER PANEL
        JPanel headerPanel = new JPanel(new GridBagLayout());
        headerPanel.setBackground(PRIMARY_BLUE);
        headerPanel.setPreferredSize(new Dimension(430, 60));

        JLabel headerLabel = new JLabel("+  ADD EMPLOYEE");
        headerLabel.setFont(new Font("Segoe UI", Font.BOLD, 18));
        headerLabel.setForeground(Color.WHITE);
        headerPanel.add(headerLabel);

        add(headerPanel, BorderLayout.NORTH);

        // 2. CENTER FORM PANEL
        JPanel centerPanel = new JPanel();
        centerPanel.setLayout(new BoxLayout(centerPanel, BoxLayout.Y_AXIS));
        centerPanel.setBorder(BorderFactory.createEmptyBorder(15, 25, 10, 25));
        centerPanel.setBackground(new Color(245, 247, 250));
```

Search Employee Interface

```
public class SearchEmployeeFrame extends JFrame {

    private JTextField idField;
    private JTextArea resultArea;
    private JButton searchButton, clearButton, backButton;
    private JLabel statusLabel;

    private final String ID_PH = "e.g., 101";

    // Colors & Borders
    private final Color PRIMARY_BLUE = new Color(30, 50, 92);
    private final Color DEFAULT_BORDER_COLOR = new Color(200, 205, 215);
    private final Color ERROR_COLOR = new Color(220, 50, 50);

    private final Border defaultBorder = BorderFactory.createCompoundBorder(
            new LineBorder(DEFAULT_BORDER_COLOR, 1),
            BorderFactory.createEmptyBorder(5, 8, 5, 8)
    );
    private final Border errorBorder = BorderFactory.createCompoundBorder(
            new LineBorder(ERROR_COLOR, 2),
            BorderFactory.createEmptyBorder(5, 8, 5, 8)
    );

    public SearchEmployeeFrame(EmployeeManager manager) {
        setTitle("Search Employee");
        setSize(430, 530);
        setLocationRelativeTo(null);
        setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
        setResizable(false);
        setLayout(new BorderLayout());

        // 1. HEADER PANEL
        JPanel headerPanel = new JPanel(new GridBagLayout());
        headerPanel.setBackground(PRIMARY_BLUE);
        headerPanel.setPreferredSize(new Dimension(430, 60));

        JLabel headerLabel = new JLabel("SEARCH EMPLOYEE");
        headerLabel.setFont(new Font("Segoe UI", Font.BOLD, 18));
        headerLabel.setForeground(Color.WHITE);
        headerPanel.add(headerLabel);

        add(headerPanel, BorderLayout.NORTH);

        // 2. CENTER PANEL
        JPanel centerPanel = new JPanel();
        centerPanel.setLayout(new BoxLayout(centerPanel, BoxLayout.Y_AXIS));
        centerPanel.setBorder(BorderFactory.createEmptyBorder(15, 25, 10, 25));
        centerPanel.setBackground(new Color(245, 247, 250));

        // Top Search Input Panel
        JPanel topSearchPanel = new JPanel(new FlowLayout(FlowLayout.CENTER, 10, 0));
        topSearchPanel.setBackground(new Color(245, 247, 250));

        JLabel idLabel = new JLabel("Enter ID:");
        idLabel.setFont(new Font("Segoe UI", Font.BOLD, 13));

        idField = createStyledTextField();
        idField.setPreferredSize(new Dimension(140, 32));
```

Update Employee Interface

```
public class UpdateEmployeeFrame extends JFrame {

    private JTextField idField, nameField, salaryField, hoursField, rateField, bonusField;
    private JComboBox<String> genderBox, typeBox;
    private JButton loadButton, updateButton, clearButton, backButton;
    private JPanel detailsPanel, dynamicPanel;
    private CardLayout cardLayout;
    private JLabel statusLabel;
    private Employee currentEmp;

    // Placeholders
    private final String ID_PH = "e.g., 101";
    private final String NAME_PH = "e.g., John Doe";
    private final String SALARY_PH = "e.g., 50000";
    private final String BONUS_PH = "e.g., 5000";
    private final String HOURS_PH = "e.g., 40";
    private final String RATE_PH = "e.g., 500";

    // Colors & Borders
    private final Color PRIMARY_BLUE = new Color(30, 50, 92);
    private final Color DEFAULT_BORDER_COLOR = new Color(200, 205, 215);
    private final Color ERROR_COLOR = new Color(220, 50, 50);

    private final Border defaultBorder = BorderFactory.createCompoundBorder(
            new LineBorder(DEFAULT_BORDER_COLOR, 1),
            BorderFactory.createEmptyBorder(5, 8, 5, 8)
    );
    private final Border errorBorder = BorderFactory.createCompoundBorder(
            new LineBorder(ERROR_COLOR, 2),
            BorderFactory.createEmptyBorder(5, 8, 5, 8)
    );

    public UpdateEmployeeFrame(EmployeeManager manager) {
        setTitle("Update Employee");
        setSize(430, 530);
        setLocationRelativeTo(null);
        setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
        setResizable(false);
        setLayout(new BorderLayout());

        // 1. HEADER PANEL
        JPanel headerPanel = new JPanel(new GridBagLayout());
        headerPanel.setBackground(PRIMARY_BLUE);
        headerPanel.setPreferredSize(new Dimension(430, 60));

        JLabel headerLabel = new JLabel("EDIT  UPDATE EMPLOYEE");
        headerLabel.setFont(new Font("Segoe UI", Font.BOLD, 18));
        headerLabel.setForeground(Color.WHITE);
        headerPanel.add(headerLabel);

        add(headerPanel, BorderLayout.NORTH);

        // 2. CENTER PANEL
        JPanel centerPanel = new JPanel();
        centerPanel.setLayout(new BoxLayout(centerPanel, BoxLayout.Y_AXIS));
        centerPanel.setBorder(BorderFactory.createEmptyBorder(15, 25, 10, 25));
        centerPanel.setBackground(new Color(245, 247, 250));
```

View Employee Interface

```
    public ViewEmployeeFrame(EmployeeManager manager) {
        setTitle("View All Employees");
        setSize(650, 420);
        setLocationRelativeTo(null);
        setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
        setLayout(new BorderLayout(10, 10));

        // Table Columns Header
        String[] columns = {"ID", "Name", "Gender", "Type", "Base Salary / Rate", "Bonus / Hours", "Total Salary"};

        tableModel = new DefaultTableModel(columns, 0) {
            @Override
            public boolean isCellEditable(int row, int column) {
                return false; // টেবিল এডিটেবল না রাখার জন্য
            }
        };

        table = new JTable(tableModel);
        table.setFont(new Font("Segoe UI", Font.PLAIN, 13));
        table.getTableHeader().setFont(new Font("Segoe UI", Font.BOLD, 13));
        table.setRowHeight(25);

        // EmployeeManager থেকে সব তথ্য নিয়ে টেবিলে যোগ করা
        for (Employee emp : manager.getEmployeeList()) {
            String salaryOrRate = "";
            String bonusOrHours = "";

            if (emp instanceof PermanentEmployee) {
                PermanentEmployee pe = (PermanentEmployee) emp;
                salaryOrRate = String.format("%.2f", pe.getBaseSalary());
                bonusOrHours = String.format("%.2f", pe.getBonus());
            } else if (emp instanceof PartTimeEmployee) {
                PartTimeEmployee pte = (PartTimeEmployee) emp;
                salaryOrRate = String.format("%.2f", pte.getRatePerHour());
                bonusOrHours = String.format("%.2f", pte.getWorkingHours());
            }

            Object[] row = {
                    emp.getId(),
                    emp.getName(),
                    emp.getGender(),
                    emp.getType(),
                    salaryOrRate,
                    bonusOrHours,
                    String.format("%.2f", emp.getTotalSalary())
            };
            tableModel.addRow(row);
        }

        JScrollPane scrollPane = new JScrollPane(table);
        add(scrollPane, BorderLayout.CENTER);
```

Fired Employee Interface

```
//============================================================
private JPanel buildMenu() {
    JPanel menu = new JPanel(new GridBagLayout());
    menu.setBackground(new Color(245, 247, 250));
    menu.setBorder(BorderFactory.createEmptyBorder(25, 40, 25, 40));

    GridBagConstraints gc = new GridBagConstraints();
    gc.gridx = 0;
    gc.fill = GridBagConstraints.HORIZONTAL;
    gc.insets = new Insets(8, 0, 8, 0);   // space between buttons
    gc.ipady = 12;                        // make buttons taller

    //==========================
    // Create the 6 buttons
    //==========================
    JButton addBtn    = makeMenuButton("+  Add Employee");
    JButton searchBtn = makeMenuButton("🔍  Search Employee");
    JButton updateBtn = makeMenuButton("✎  Update Employee");
    JButton viewBtn   = makeMenuButton("📋  View All Employees");
    JButton deleteBtn = makeMenuButton("🗑  Fired  Employee");
    JButton exitBtn   = makeMenuButton("⏻  Exit");

    //==========================
    // Button actions (open the other windows)
    //==========================
    addBtn.addActionListener(e -> new AddEmployeeFrame(manager).setVisible(true));
    searchBtn.addActionListener(e -> new SearchEmployeeFrame(manager).setVisible(true));
    updateBtn.addActionListener(e -> new UpdateEmployeeFrame(manager).setVisible(true));
    viewBtn.addActionListener(e -> new ViewEmployeeFrame(manager).setVisible(true));
    deleteBtn.addActionListener(e -> new DeleteEmployeeFrame(manager).setVisible(true));
    exitBtn.addActionListener(e -> System.exit(0));

    //==========================
    // Add the buttons one by one
    //==========================
    gc.gridy = 0; menu.add(addBtn, gc);
    gc.gridy = 1; menu.add(searchBtn, gc);
    gc.gridy = 2; menu.add(updateBtn, gc);
    gc.gridy = 3; menu.add(viewBtn, gc);
    gc.gridy = 4; menu.add(deleteBtn, gc);
    gc.gridy = 5; menu.add(exitBtn, gc);

    return menu;
}
```

2.2 Overall Project Plan

Phase 1 (Setup & OOP Architecture): Implement the core abstract class and its subclasses, and initialize an ArrayList to support O(1) append operations. Phase 2 (Core Operations): Develop backend CRUD operations (add, search, update, delete) within the EmployeeManager using O(n) traversal. Phase 3 (GUI & Advanced Features): Build a Java Swing UI using CardLayout and GridBagLayout, and add strict inline input validation with dynamic type conversion logic. Phase 4 (Integration): Connect all independent frontend GUI frames to the shared backend manager to enable real-time state synchronization. Phase 5 (Testing & Optimization)**:** Fix UI alignment stretching, handle exceptions (e.g., NumberFormatException) to prevent crashes, and verify data integrity constraints. Phase 6 (Documentation): Complete the comprehensive project report detailing OOP principles, system workflow, and architectural design.

Chapter 3

Implementation and Results

This chapter details the practical implementation of the Employee Management System (EMS). It explains the development environment, the translation of object-oriented concepts into working code, the execution of dynamic graphical interfaces, and the advanced validation logic implemented to ensure structural and data integrity.

3.1 Implementation Environment

The system was developed as a standalone desktop application without reliance on heavy external frameworks. The development environment consists of:

- Programming Language: Core Java (JDK 8 or higher), chosen for its robust Object-Oriented features and cross-platform portability.
- GUI Framework: Java Swing and Abstract Window Toolkit (AWT), utilized to build lightweight, native-looking desktop windows and interactive form elements.
- Development IDE**:** Modern Java IDEs (such as IntelliJ IDEA / Eclipse / VS Code) were used for writing, compiling, and debugging the source code.
- Data Storage**:** Ephemeral in-memory storage using standard Java Collections (ArrayList), avoiding the overhead of configuring external database servers like MySQL or SQLite.

3.2 Core System Implementation

The core architecture translates the theoretical design from Chapter 2 into functional Java code. The backend logic and frontend views are carefully separated to maintain a clean codebase.

3.2.1 Object-Oriented Implementation (Polymorphism & Inheritance)

The core logic relies heavily on OOP principles. An abstract parent class named Employee was created to hold shared attributes (ID, Name, Gender) and define abstract methods like getTotalSalary(). From this base class, two distinct subclasses inherit the properties:

- PermanentEmployee: Adds specific attributes for baseSalary and bonus.
- PartTimeEmployee: Adds specific attributes for workingHours and ratePerHour. This hierarchical design allows the system to process both employee types uniformly via Polymorphism while calculating their respective salaries using different internal mathematical formulas.

3.2.2 In-Memory Data Management (ArrayList CRUD)

Instead of using SQL databases, the EmployeeManager class acts as the centralized data engine. It utilizes an ArrayList<Employee> to store active session data. This approach allows the application to execute CRUD operations efficiently:

- Create**:** Appending new employee objects to the list in O(1) time complexity.
- Read & Update: Traversing the list using loops (O(n) complexity) to search for specific IDs and modify their attributes in real-time.
- Delete: Instantly removing objects from the memory index securely.

3.2.3 Dynamic GUI Layouts

(GridBagLayout & CardLayout**)** To ensure the application looks professional and scales perfectly on screen, advanced Swing layout managers were utilized:

- GridBagLayout**:** Used for precise, grid-based alignment of labels and text fields, ensuring perfectly uniform vertical and horizontal gaps (insets) between form elements.
- CardLayout**:** Implemented to create dynamic, context-aware forms. When an administrator selects "Permanent" or "Part-Time" from a dropdown box (JComboBox), the CardLayout instantly swaps the bottom input fields (e.g., hiding "Hours Worked" and showing "Basic Salary") without requiring a new window to open.

3.3 Advanced Logic & Validation

To simulate a production-ready application, several advanced safeguards and formatting logic were implemented at the code level to prevent crashes and ensure absolute data integrity.

3.3.1 Strict Input Validation & Regex:

The system actively prevents administrative errors through deep logical checks. Regular Expressions (Regex: ^[a-zA-Z\\s.\\-]+$) were implemented to ensure the "Name" field strictly accepts alphabetic characters, spaces, dots, and hyphens—instantly rejecting numbers or special symbols. Furthermore, strict mathematical boundaries were added to block negative values (e.g., negative salaries, negative working hours, or zero IDs).

3.3.2 Exception Handling and Inline Visual Feedback

Instead of interrupting the user experience with constant error pop-up dialogs, the system uses modern inline validation. try-catch blocks are used to catch NumberFormatException when a user mistakenly types letters into numeric fields. When an error is caught, the system dynamically changes the specific text field's border color to red and displays a clear [!] warning message directly within the UI, forcing the user to provide valid data before proceeding.

3.3.3 Dynamic Type Switching Capabilities

A highly advanced feature implemented in the UpdateEmployeeFrame is "Type Conversion". If a Part-Time employee is promoted to a Permanent role, the system does not just update text fields; it securely deletes the old PartTimeEmployee object from the ArrayList and injects a brand-new PermanentEmployee object in the backend while preserving the original Employee ID.

3.3.4 Perfect UI Formatting (Monospaced Alignment)

To ensure output data looks structurally perfect, especially in the SearchEmployeeFrame, a combination of Consolas (Monospaced font) and Java's String.format("%-15s: %s\n") was utilized. Since monospaced fonts assign the same pixel width to every character, the %-15s padding guarantees that all the colons (:) align in a perfectly straight, scale-like vertical line, significantly improving the visual readability of the output data.

3.4 System Results & Interfaces

Figure 3.4.1: Main Dashboard Interface

Figure 3.4.2: Adding a New Employee

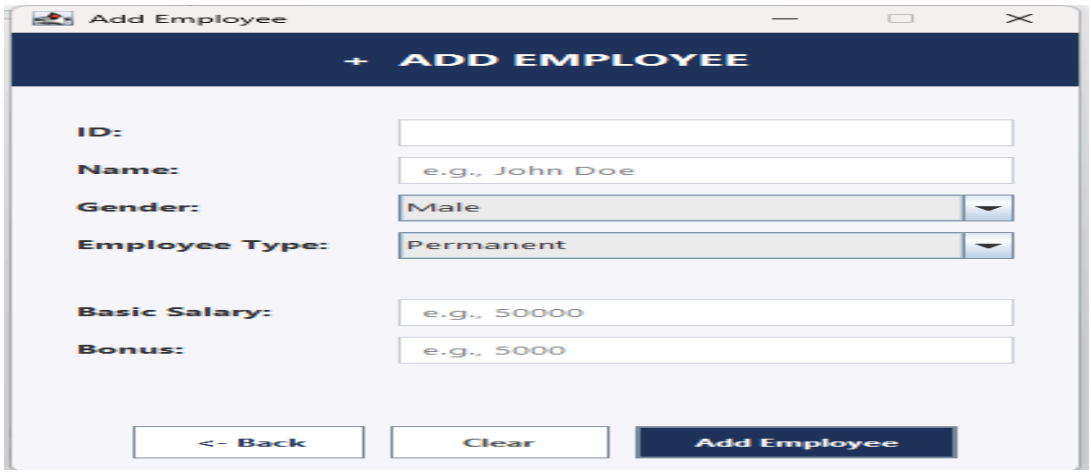


Figure 3.4.3: Real-time Error Handling

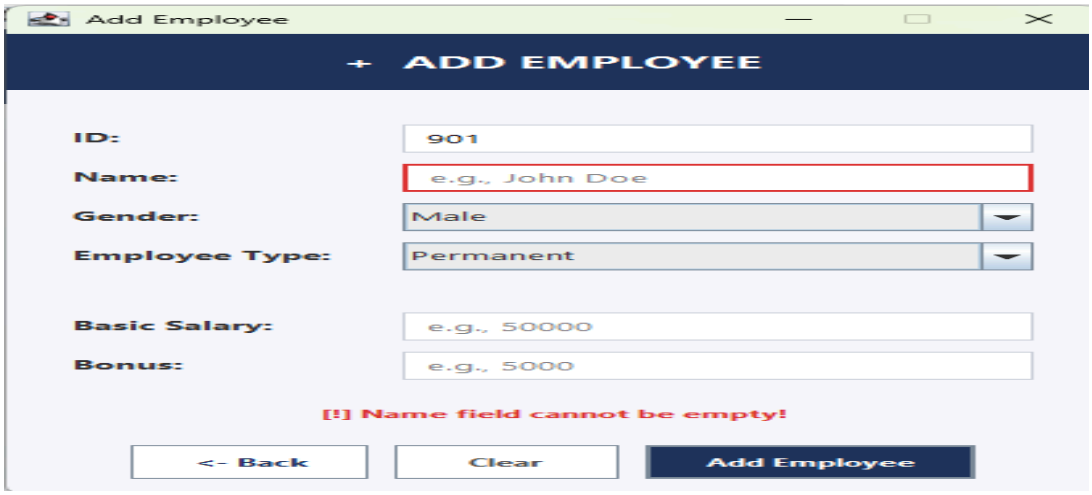


Figure 3.4.4: Searching and Viewing Employee Details

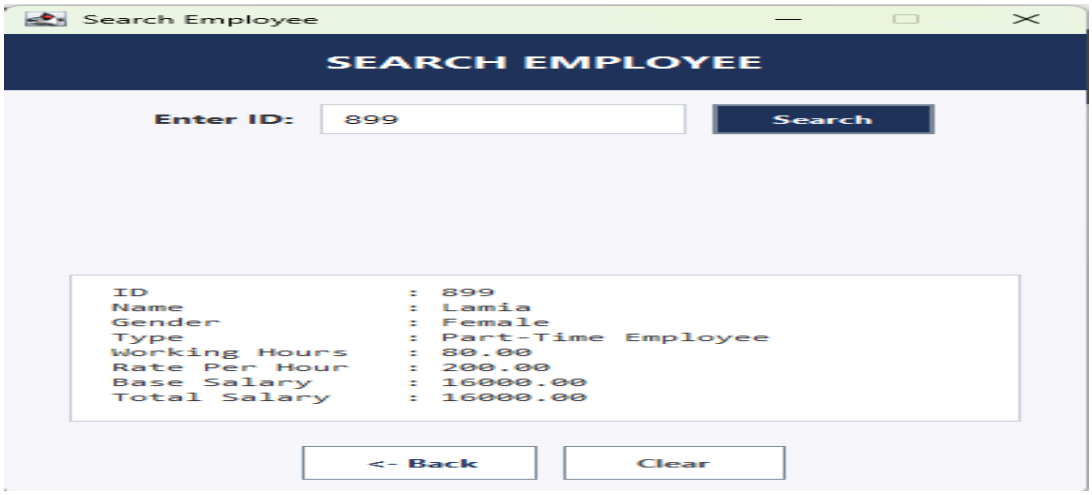


Figure 3.4.5: Updating Employee Records and Type Switch

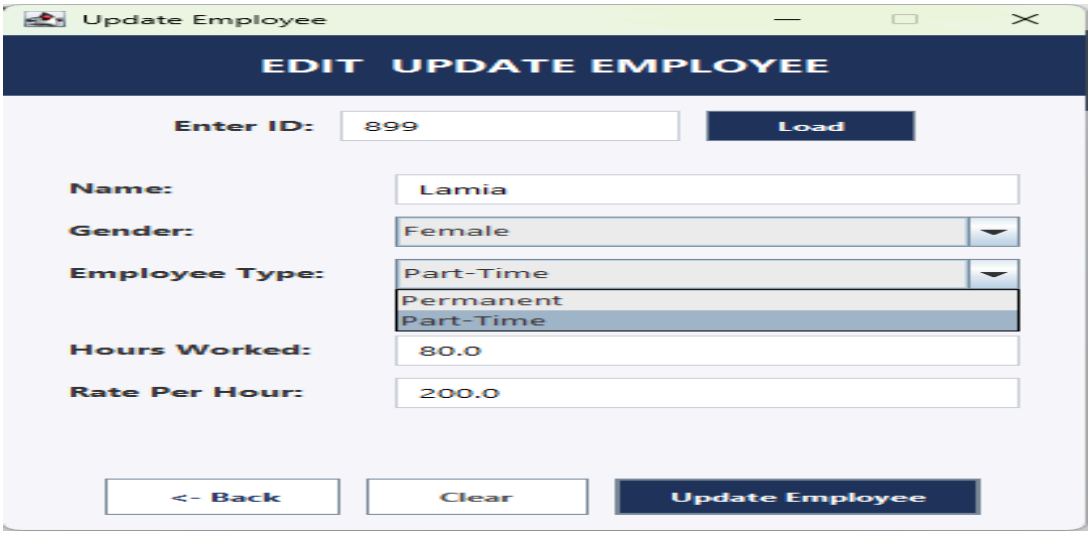
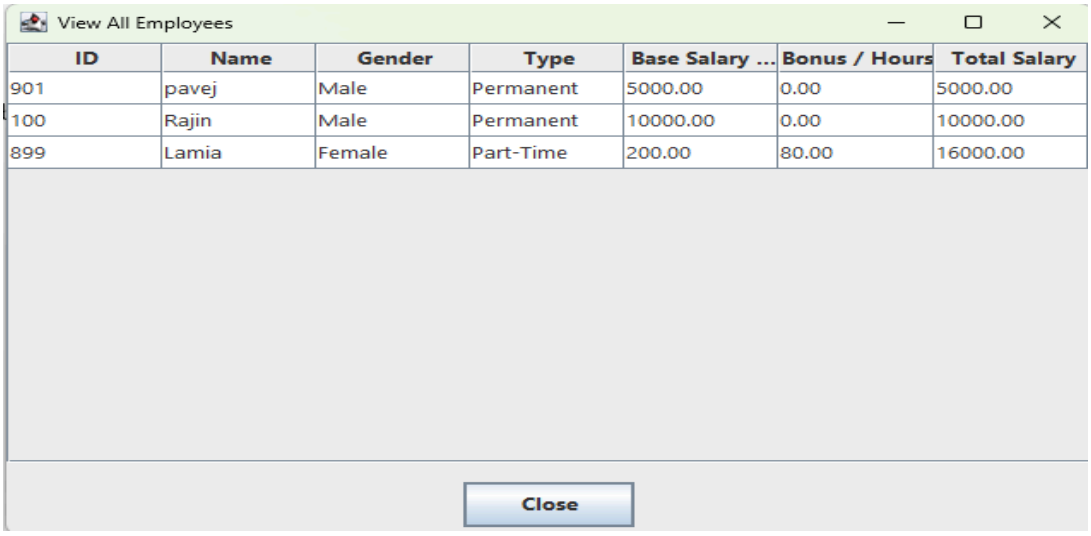


Figure 3.4.6: Tabular View of All Employees



Chapter 4

Engineering Standards and Mapping

This chapter maps the project to program outcomes, engineering standards, complex problem-solving categories, and engineering activities, demonstrating alignment with educational objectives.

4.1 Impact on Society, Environment and Sustainability

4.1.1 Impact on Life

This Employee Management System improves the quality of professional life by automating administrative tasks and reducing manual errors. Accurate, automated salary calculations based on employee roles (Permanent or Part-Time) ensure fair compensation, directly impacting employee satisfaction and financial stability.

4.1.2 Impact on Society & Environment

By transitioning from traditional paper-based HR records to a digital platform, the system significantly reduces paper waste and physical storage needs, supporting environmental conservation. Its lightweight, in-memory architecture (ArrayList) consumes minimal computational resources, making digital management accessible and eco-friendly for small to medium-sized businesses.

4.1.3 Ethical Aspects

The system prioritizes data integrity, fairness, and transparency. Strict input validation prevents the entry of corrupted or invalid data (e.g., negative salaries). Automated computations eliminate human bias in payroll management. The structured CRUD operations ensure that employee records are managed systematically without unauthorized data manipulation.

4.1.4 Sustainability Plan

The system's modular Object-Oriented Programming (OOP) architecture ensures long-term sustainability. In the future, the in-memory storage can be seamlessly upgraded to a relational database (like MySQL) without altering the frontend UI. The clean, well-documented codebase allows future developers to easily scale the application to include cloud backups, web interfaces, or mobile applications.

4.2 Project Management and Teamwork

The project was completed through effective teamwork by dividing responsibilities among all team members. Each member contributed to different parts of the system based on their strengths and understanding.

Two members were mainly responsible for developing the backend core program, implementing the major OOP functionalities, ArrayList CRUD operations, and polymorphic salary logic. One member focused heavily on designing the Java Swing user interface (utilizing CardLayout and GridBagLayout) to ensure the system is modern, responsive, and easy to use. Another member worked on preparing the project report by organizing the content, maintaining proper structure, and ensuring clarity. One member was dedicated to debugging, rigorously testing the system for UI alignment issues and input errors (like NumberFormatException), and fixing them to improve overall performance.

Although the tasks were divided, all members worked together collaboratively throughout the project. The system was tested multiple times, and several bugs occurred during the integration of the frontend and backend. However, through continuous effort, group discussion, and teamwork, these issues were identified and resolved successfully. Every member contributed actively, and the project was improved step-by-step through repeated testing and correction, ensuring all components functioned perfectly.

4.3 Complex Engineering Problem

4.3.1 Program Outcome Mapping

Table 4.1: Justification of Program Outcomes

PO's	Justification
PO1	Project applies core OOP principles (Abstraction, Polymorphism) and dynamic data structures (ArrayList for O(1) append, O(n) search) for efficient memory management, demonstrating engineering knowledge of algorithmic implementation and software design.
PO2	Addresses data integrity and validation problems by implementing strict Regular Expressions (Regex) and exception handling (NumberFormatException), ensuring crash-free operations and dynamic type switching without external databases.
PO3	Complete software solution: requirements analysis, modular architecture separating backend logic (EmployeeManager) from frontend GUI, full CRUD implementation, and interactive layout design (CardLayout, GridBagLayout).
PO5	Utilized modern software engineering tools including standard Java Development Kit (JDK) libraries, Java Swing toolkit for graphical interfaces, and modern IDEs for structured coding, compiling, and debugging.
PO9	Demonstrated effective teamwork by dividing tasks into backend logic development, UI/UX layout design, rigorous bug testing, and technical documentation, resulting in a cohesive and fully integrated application.

4.3.2 Complex Problem Solving Table

4.2: Complex Problem Solving Categories

EP1 Depth of Knowledge	EP2 Range of Conflicting Requirements	EP3 Depth of Analysis	EP6 Extent of Stakeholder Involvement
✔	✔	✔	✔

Table 4.2: Mapping with complex problem solving.

Category	Level	Justification
EP1 Depth of Knowledge	**High**	The project demonstrates a strong grasp of core OOP principles (Abstraction, Inheritance, Polymorphism) and dynamic memory management. By implementing an in-memory CRUD system using Java ArrayList (O(1) append, O(n) search) and advanced GUI frameworks (CardLayout, GridBagLayout), the system showcases solid engineering knowledge in software design.
EP2 Range of Conflicting Requirements	**Medium**	The system addresses conflicting requirements by balancing user experience (UX) with strict data integrity. It resolves UI space limitations by utilizing dynamic CardLayout switching to prevent screen clutter, and balances rapid data execution with minimal resource consumption using an in-memory ArrayList architecture.
EP3 Depth of Analysis	**High**	The system incorporates deep logical analysis to maintain data integrity. It uses Regular Expressions (Regex) for strict string validation, mathematical constraints to prevent negative financial inputs (salaries/hours), and dynamic backend object replacement to handle employee role changes seamlessly.
EP6 Extent of Stakeholder Involvement	**Medium**	The architecture considers the distinct needs of multiple administrative stakeholders. It provides HR/Admins with unified dashboards for quick searching and dynamic calculations, while managing two different employee groups (Permanent and Part-Time) with separate compensation structures tailored to specific operational requirements.

4.3.3 Engineering Activities Table

4.3: Engineering Activities Mapping

EA1 Range of resources	**EA2** Level of Interaction	**EA3** Innovation	**EA4** Consequences for society and environment	**EA5** Familiarity

Chapter 5

Conclusion

This chapter summarizes project accomplishments, discusses limitations encountered during implementation, highlights the debugging and problem-solving process, and presents opportunities for future development.

5.1 Summary

The Employee Management System (EMS) has been successfully implemented, demonstrating the practical application of Object-Oriented Programming (OOP) concepts and a dynamic Java Swing GUI. The system efficiently integrates an ArrayList for in-memory data management, exhibiting time complexities of O(1) for appending new records and O(n) for searching, updating, and deleting operations. All planned functional features were implemented effectively: polymorphic handling of employee types (Permanent and Part-Time), dynamic layout switching (CardLayout and GridBagLayout), strict real-time input validation (Regex and Exception Handling), advanced dynamic type conversion, and perfectly aligned textual data retrieval.

5.2 Limitations

The current system has a few limitations due to the educational scope and architectural design choices of the project:

- No Persistent Data Storage: The system currently uses an in-memory ArrayList. Since there is no database (SQL) or file-based storage integrated, all employee records are lost once the application is closed.
- Time Complexity in Searching (O(n)): Searching, updating, or deleting an employee by ID requires traversing the ArrayList sequentially. While extremely fast for small-scale applications, this O(n) complexity could slow down performance if handling hundreds of thousands of records.
- Single-User Desktop Environment: The application runs locally as standalone desktop software. It lacks a network-based multi-user environment or role-based authentication (Login system), meaning anyone with access to the computer can modify the records.
- Lack of Encryption: Sensitive financial data, such as basic salary, hourly rates, and bonuses, are handled in plain text within the system memory without cryptographic encryption.

■ Debugging and Problem Solving

During the development of the system, several critical issues were identified and resolved through rigorous testing and debugging. These solutions significantly improved the system’s reliability and UI/UX functionality:

- Numeric Name Loophole: Initially, the system's "Name" field accepted numbers (e.g., "45"). This logical vulnerability was identified and fixed by implementing a strict Regular Expression (^[a-zA-Z\\s.\\-]+$) to ensure only valid alphabetic characters and standard punctuation are accepted.

- Dynamic Layout Stretching Bug: In the UpdateEmployeeFrame, switching between Permanent and Part-Time employee types caused the input fields to stretch vertically and create large, unpleasant gaps. This UI bug was resolved by utilizing GridBagLayout with exact 12px insets and implementing Box.createVerticalGlue() to push all components cleanly to the top.
- Output Alignment Issue: When searching for an employee, the colons (`:`) in the details view were misaligned due to varying character widths in default fonts. This was debugged and fixed by applying a monospaced font (Consolas) combined with precise string padding (String.format("%-15s: %s\n")), achieving 100% straight vertical alignment.
- Type Conversion Data Conflict: Updating an employee's role from Part-Time to Permanent initially caused data mapping conflicts. This was solved by designing backend logic that securely deletes the old subclass object and injects a brand-new subclass object while preserving the original Employee ID.

5.3 Future Work

1. Database Integration: Replace the in-memory ArrayList with a relational database (e.g., MySQL or PostgreSQL) to achieve permanent data persistence and O(1) indexed queries.
2. Optimize Search Operations: Implement a HashMap<Integer, Employee> to reduce search complexity from O(n) to O(1) average time for instant ID lookups.
3. Authentication Mechanism: Add a secure Login screen with Role-Based Access Control (Admin vs. Standard User) to restrict unauthorized data modifications.
4. Data Exporting Feature: Integrate external libraries to generate and export monthly salary slips or employee records into PDF or CSV formats.
5. File I/O Implementation: Introduce binary serialization or JSON file saving as a lightweight alternative to a full database for session data retention.
6. Advanced Security: Add AES encryption to secure sensitive financial data (salaries and bonuses) while the application is running.

References

1. Kavanagh, Michael J., and Richard D. Johnson. *Human Resource Information Systems: Basics, Applications, and Future Directions*. SAGE Publications, 2020.
2. Schildt, Herbert. *Java: The Complete Reference*. McGraw-Hill Education, 2018.
3. Goodrich, Michael T., Roberto Tamassia, and Michael H. Goldwasser. *Data Structures and Algorithms in Java*. John Wiley & Sons, 2014.
4. https://docs.oracle.com/javase/tutorial/
5. https://www.geeksforgeeks.org/

Report Contribution

Name	ID	Contribution
Md Parvej	**251-15-901**	Project report, diagram, Debugging, UI implementation, Limitation finding.
Md Abu Salih	**251-15-100**	PPT Slide Preparation
Anik Shil Antu	**251-15-667**	Program & QA Testing
Imran	**251-15-575**	Idea Generator
Yeasin Arafat	**251-15-914**	Requirement Gathering